Eötvös Loránd University

Databases and data models

Project I: Oracle Movie Database

Instructor: Péter Lehotay-Kéry Dr.

Student: Naghi Naghiyev

NEPTUN ID: HKMNUM

Project Overview

This is a project for a relational database on Oracle for a movie domain. The purpose is to create a database schema, import a realistic volume of data, and demonstrate Oracle functionality such as partitioning, views, indexes, PL/SQL, a sequence and trigger. The data has been generated artificially, created with Python scripts, rather than loaded from an external source. The generated CSV files contain information on directors, actors, movies, cast, users, and reviews. The database development process is well structured, starting with creating tables in SQL, then importing the CSV data into Oracle, and finally creating additional database objects such as views, indexes, PL/SQL, a sequence, a trigger, and privileges for a second database user.

Process Description

I did the given tasks with the following order:

1. Find a theme that I can generate data for (in my case it is movies).
2. Decided how the tables and the relationship between them must be.
3. Wrote python scripts that will generate the csv tables for my database.
4. Run the python codes, check the generated data's quality and distribution.
5. Created initial empty tables using SQL queries.
6. Imported the csv files to those created tables.
7. Fulfill the sequence and trigger requirement. Created a new table.
8. Implemented indexes.
9. Created views for some of the tables.
10. Implemented PL/SQL function and procedure.
11. Give grants to the instructor.

Phase 1: Data Generation

For data generation, I chose Python as it has large libraries and capabilities to generate synthetic data. For example, the Faker library is specifically designed for artificial data generation, and it was the main library that I used during this phase.
I decided that there must be six tables:

1. directors 1,000 rows × 3 cols
2. actors 5,000 rows × 4 cols
3. users 50,000 rows × 4 cols
4. movies 10,000 rows × 6 cols
5. roles 200,000 rows × 3 cols
6. reviews 150,000 rows × 5 cols

I wrote one Python script per table and combined them under one generation script. That script runs each generator in order (respecting foreign-key dependencies between entities) and produces all of the CSV files. Figure 1 shows terminal output from a full run, including per-step timings and the summary.

Figure 1: Output of the combined data generation script.

After generation, I use a test script to verify that the following hold for every table:

1. Row and column counts match the specification.
2. No column contains NULL values.
3. There are no duplicate rows.
4. The ranges and constraints for columns such as `birthYear` or `releaseDate` are satisfied.
5. Primary and foreign keys are consistent.

6. The data distribution matches what was described above.

This completes Phase 1. The resulting CSV files are the input for table creation and loading in Phase 2.

Phase 2: Table Creation

The CSV files are ready to use. I created empty tables in SQL on the Aramis database so that the data can be loaded afterward. The DDL is listed in referential order: dimension tables first, then `movies`, then bridge and fact-like tables. Each block below is the exact statement used for one object; SQL comments from the source script were dropped and replaced by the prose before each listing.

Directors. The `directors` table holds one row per director: a numeric primary key, the display name, and a four-digit birth year. All columns are mandatory so that every movie can resolve a complete director record.

```
CREATE TABLE directors (
  directorID NUMBER(6) PRIMARY KEY,
  directorName VARCHAR2(24) NOT NULL,
  birthYear NUMBER(4) NOT NULL
);
```

Actors. The `actors` table stores cast members with the same key and birth-year pattern as directors, plus a short nationality string for variety in reporting.

```
CREATE TABLE actors (
  actorID NUMBER(6) PRIMARY KEY,
  actorName VARCHAR2(28) NOT NULL,
  birthYear NUMBER(4) NOT NULL,
  nationality VARCHAR2(16) NOT NULL
);
```

Users. The `users` table models registered accounts: identifier, username, country, and registration date. It is independent of the movie catalog and must exist before reviews that reference users.

```
CREATE TABLE users (
  userID NUMBER(6) PRIMARY KEY,
  username VARCHAR2(24) NOT NULL,
  country VARCHAR2(52) NOT NULL,
  registrationDate DATE NOT NULL
);
```

Movies. The `movies` table links each title to a director via a foreign key on `directorID`. Release year, genre, and title are required. `duration` is constrained to be strictly positive.

```
CREATE TABLE movies (
  movieID NUMBER(6) PRIMARY KEY,
  movieTitle VARCHAR2(36) NOT NULL,
  releaseYear NUMBER(4) NOT NULL,
  genre VARCHAR2(12) NOT NULL,
  directorID NUMBER(6) NOT NULL REFERENCES directors(directorID),
  duration NUMBER(3) NOT NULL CHECK (duration > 0)
);
```

Roles. The `roles` table is the many-to-many link between movies and actors. Each row names one part in one film; the primary key is the pair (`movieID`, `actorID`), and both sides reference their parent tables.

```
CREATE TABLE roles (
  movieID NUMBER(6) NOT NULL REFERENCES movies(movieID),
  actorID NUMBER(6) NOT NULL REFERENCES actors(actorID),
  roleName VARCHAR2(28) NOT NULL,
  CONSTRAINT pkRoles PRIMARY KEY (movieID, actorID)
);
```

Reviews (table and partitions). The `reviews` table stores one rating per user and movie, with a synthetic `reviewID` and a calendar `reviewDate`. Ratings are restricted to the closed interval $[0, 10]$. The table is range-partitioned on `reviewDate` into five slices: four five-year windows from 2005 upward and a final partition for `MAXVALUE`, so that large review volumes can be pruned or maintained by time range.

```
CREATE TABLE reviews (
    reviewID NUMBER(6) NOT NULL,
    movieID NUMBER(6) NOT NULL REFERENCES movies(movieID),
    userID NUMBER(6) NOT NULL REFERENCES users(userID),
    rating NUMBER(2) NOT NULL CHECK (rating BETWEEN 0 AND 10),
    reviewDate DATE NOT NULL
)
PARTITION BY RANGE (reviewDate) (
    PARTITION reviews_2005_2010 VALUES LESS THAN (TO_DATE('2010-01-01','YYYY
        -MM-DD')),
    PARTITION reviews_2010_2015 VALUES LESS THAN (TO_DATE('2015-01-01','YYYY
        -MM-DD')),
    PARTITION reviews_2015_2020 VALUES LESS THAN (TO_DATE('2020-01-01','YYYY
        -MM-DD')),
    PARTITION reviews_2020_2025 VALUES LESS THAN (TO_DATE('2025-01-01','YYYY
        -MM-DD')),
    PARTITION reviews_2025_now VALUES LESS THAN (MAXVALUE)
);
```

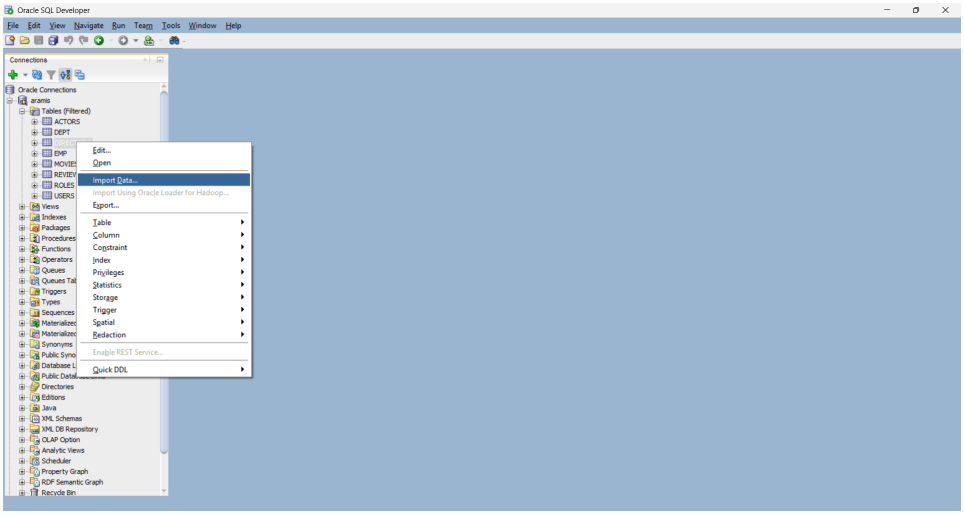
Reviews (primary key and local index). Because Oracle requires the partition key in a unique or primary key on a partitioned table, the primary key is (`reviewID`, `reviewDate`). The constraint is added in a second step with `USING INDEX LOCAL` so that the index is equipartitioned with the table, which keeps maintenance aligned with partitions.

```
ALTER TABLE reviews ADD CONSTRAINT reviewsPk PRIMARY KEY (reviewID,
    reviewDate) USING INDEX LOCAL;
```

With the schema in place on aramis, the next step is to load the CSV files produced in Phase 1 into these tables. Phase 2 ends here; Phase 3 describes that loading process.

Phase 3: Data Loading

After creating the tables, I imported the generated CSV files using Oracle SQL Developer user interface. Files were loaded with order: `directors` and `actors` first (no dependencies on other project tables), then `users`, then `movies` (which references `directors`), then `roles` (which references `movies` and `actors`), and finally `reviews` (which references `movies` and `users`). I imported data by following steps: right click on the table, click import data, browse CSV file, follow the instructions and import the data.



Phase 4: E-R Diagram and Data Model

I drew an ER diagram for this database (Figure 2). It shows the tables, columns, and keys. The `movie_audit` table is on the diagram too because I add it in a later step.

Roughly: one director has many movies; one movie has many reviews and many cast lines in `roles`; one user has many reviews. Actors and movies meet in `roles` (many-to-many). `reviews` needs a primary key on (`reviewID`, `reviewDate`) because of partitioning. `movie_audit` is drawn next to `movies` using `movieID`, but I did not add a real foreign key there in SQL—it is only for the picture.
Why does `roles` have PK and FK on the same columns? The pair (`movieID`, `actorID`) is the primary key so each movie–actor pair appears once. Each id also has to exist in `movies` or `actors`, so they are foreign keys too. The diagram marks both.

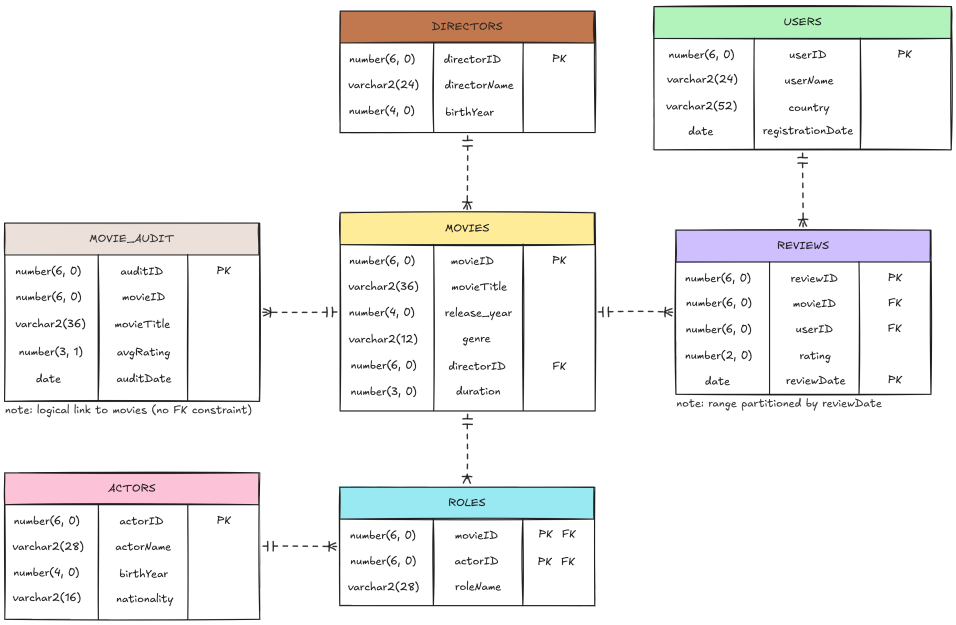


Figure 2: Entity–relationship diagram of the movie database.

Phase 5: Sequence Trigger

My first tables already had ids inside the CSV files, so a sequence was not really needed there. The task still asks for a sequence and a trigger, so I made a new table called `movie_audit`. It stores one audit line per movie: `auditID`, `movieID`, `movieTitle`, `avgRating`, and `auditDate`. The rating column must be between 0 and 10. `auditDate` fills with the current date if I do not pass a value (`SYSDATE`).
The script `sequencetrigger.sql` creates the table, then a sequence `auditSeq` that counts 1, 2, 3, ... (`START WITH 1`, `INCREMENT BY 1`). After that comes a trigger `auditBeforeInsert` on `movie_audit`. It runs before each insert:

if `auditID` is null, it pulls the next number from the sequence so I do not have to type the id by hand.

```
CREATE TABLE movie_audit (
    auditID NUMBER(6) PRIMARY KEY,
    movieID NUMBER(6) NOT NULL,
    movieTitle VARCHAR2(36) NOT NULL,
    avgRating NUMBER(3,1) NOT NULL CHECK (avgRating BETWEEN 0 AND 10),
    auditDate DATE DEFAULT SYSDATE NOT NULL
);
```

```
CREATE SEQUENCE auditSeq
    START WITH 1
    INCREMENT BY 1;
```

```
CREATE OR REPLACE TRIGGER auditBeforeInsert
BEFORE INSERT ON movie_audit
FOR EACH ROW
BEGIN
    IF :NEW.auditID IS NULL THEN
        SELECT auditSeq.NEXTVAL INTO :NEW.auditID FROM dual;
    END IF;
END;
/
```

Figure 3 shows the script in SQL Developer and the messages after I ran it: the table, sequence, and trigger all built without errors.

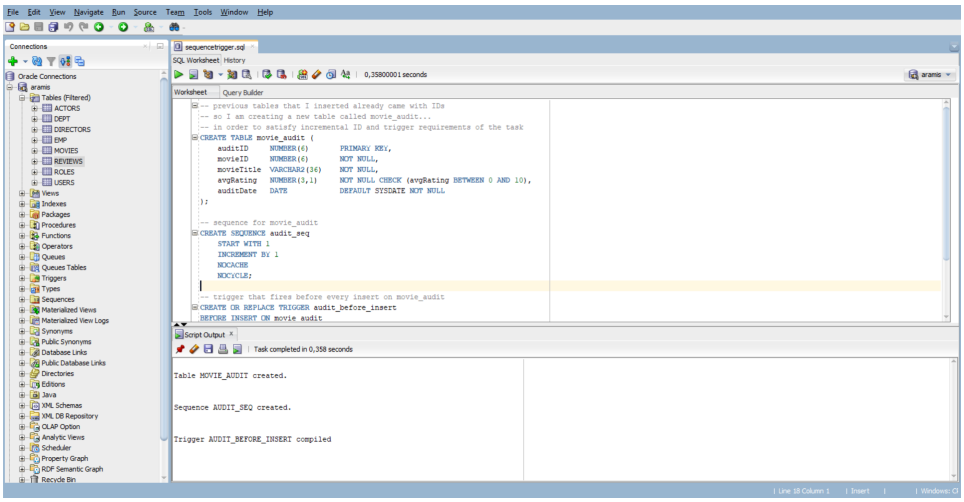


Figure 3: Running `sequencetrigger.sql` in SQL Developer (script output).

Phase 6: Indexes

In this phase I added indexes to some columns that are often used in joins or filters. Without indexes, Oracle has to scan entire tables to find matching rows, which is slow when tables are large. An index lets Oracle jump directly to the rows it needs. First I ran an execution plan on a query that joins `movies`, `directors`, and `reviews` and filters by a director name, before creating any indexes. The plan showed a cost of 1389 and used full table scans on all three tables.

```
EXEC DBMS_STATS.GATHER_SCHEMA_STATS(USER);
EXPLAIN PLAN FOR
    SELECT m.movieTitle, d.directorName, COUNT(r.reviewID) AS totalReviews
    FROM movies m
    JOIN directors d ON m.directorID = d.directorID
    JOIN reviews r ON m.movieID = r.movieID
    WHERE d.directorName = 'Nicole Mendez'
    GROUP BY m.movieTitle, d.directorName;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

```
PLAN_TABLE_OUTPUT
Plan hash value: 2553402740

-----------------------------------------------------------------------------------------------------
| Id  | Operation                | Name      | Rows  | Bytes | Cost (%CPU)| Time     | Pstart| Pstop |
-----------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |           |   151 |  5889 |  1389   (1)| 00:00:01 |       |       |
|   1 |  HASH GROUP BY           |           |   151 |  5889 |  1389   (1)| 00:00:01 |       |       |
|*  2 |   HASH JOIN              |           |   151 |  5889 |  1388   (1)| 00:00:01 |       |       |
|   3 |    TABLE ACCESS FULL     | MOVIES    | 10000 |   253K|    19   (0)| 00:00:01 |       |       |
|   4 |    MERGE JOIN CARTESIAN  |           |   150K|  1904K|  1369   (1)| 00:00:01 |       |       |
|   5 |     VIEW                 | VW_GBF_17 |     1 |     8 |     3   (0)| 00:00:01 |       |       |
|*  6 |      TABLE ACCESS FULL   | DIRECTORS |     1 |    20 |     3   (0)| 00:00:01 |       |       |
|   7 |     BUFFER SORT          |           |   150K|   732K|  1369   (1)| 00:00:01 |       |       |
|   8 |      PARTITION RANGE ALL|           |   150K|   732K|  1366   (1)| 00:00:01 |     1 |     5 |
|   9 |       TABLE ACCESS FULL | REVIEWS   |   150K|   732K|  1366   (1)| 00:00:01 |     1 |     5 |
-----------------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("M"."MOVIEID"="R"."MOVIEID" AND "M"."DIRECTORID"="ITEM_1")
   6 - filter("D"."DIRECTORNAME"='Nicole Mendez')
```

Figure 4: Execution plan before indexes. Cost is 1389.

Then I created five indexes on the columns that appear in join conditions and `WHERE` clause.

```
CREATE INDEX idxReviewsMovieId ON reviews(movieID);
CREATE INDEX idxReviewsUserId ON reviews(userID);
CREATE INDEX idxMoviesDirectorId ON movies(directorID);
CREATE INDEX idxRolesActorId ON roles(actorID);
CREATE INDEX idxDirectorsName ON directors(directorName);
```

The first index is on `reviews.movieID` because reviews is the largest table and it is joined to movies on that column. The second is on `reviews.userID` for similar reasons. The third is on `movies.directorID` so the join from directors to movies is fast. The fourth is on `roles.actorID` for the actor join. The fifth is on `directors.directorName` because the query filters on that column directly.

After creating the indexes I ran the same explain plan. The cost dropped to 24 and Oracle switched from full table scans to index range scans and nested loops.

```
EXEC DBMS_STATS.GATHER_SCHEMA_STATS(USER);
EXPLAIN PLAN FOR
    SELECT m.movieTitle, d.directorName, COUNT(r.reviewID) AS totalReviews
    FROM movies m
    JOIN directors d ON m.directorID = d.directorID
    JOIN reviews r ON m.movieID = r.movieID
    WHERE d.directorName = 'Nicole Mendez'
    GROUP BY m.movieTitle, d.directorName;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

```
PLAN_TABLE_OUTPUT
Plan hash value: 270206970

-----------------------------------------------------------------------------------------------------------
| Id  | Operation                              | Name                  | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT                       |                       |   151 |  7701 |    24   (5)| 00:00:01 |
|   1 |  HASH GROUP BY                         |                       |   151 |  7701 |    24   (5)| 00:00:01 |
|   2 |   NESTED LOOPS                         |                       |   151 |  7701 |    23   (0)| 00:00:01 |
|   3 |    NESTED LOOPS                        |                       |    10 |   460 |    13   (0)| 00:00:01 |
|   4 |     TABLE ACCESS BY INDEX ROWID BATCHED| DIRECTORS             |     1 |    20 |     2   (0)| 00:00:01 |
|*  5 |      INDEX RANGE SCAN                  | IDX_DIRECTORS_NAME    |     1 |       |     1   (0)| 00:00:01 |
|   6 |     TABLE ACCESS BY INDEX ROWID BATCHED| MOVIES                |    10 |   260 |    11   (0)| 00:00:01 |
|*  7 |      INDEX RANGE SCAN                  | IDX_MOVIES_DIRECTORID |    10 |       |     1   (0)| 00:00:01 |
|*  8 |    INDEX RANGE SCAN                    | IDX_REVIEWS_MOVIEID   |    15 |    75 |     1   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   5 - access("D"."DIRECTORNAME"='Nicole Mendez')
   7 - access("M"."DIRECTORID"="D"."DIRECTORID")
   8 - access("M"."MOVIEID"="R"."MOVIEID")

Note
-----
   - this is an adaptive plan
```

Figure 5: Execution plan after indexes. Cost dropped to 24.

The cost went down from 1389 to 24, which is a big improvement. Oracle now uses `IDXDIRECTORSNAME` to find the director row, `IDXMOVIESDIRECTORID` to get the matching movies, and `IDXREVIEWSMOVIEID` to get the reviews for each movie.

Phase 7: Views

In this phase I created two views. A view is like a saved query. It does not store data, it just runs the query on the background every time you use it. The script `views.sql` creates both views and then selects from each one to display the results.

View 1: viewMovieDetails. This view joins `movies`, `directors`, and `reviews` together. It shows the movie id, title, genre, release year, director name, and the average rating. I added a `WHERE` filter to only display movies released in 2015 or later. The rows are grouped by movie so each movie gets one row with its average rating.

```
CREATE OR REPLACE VIEW viewMovieDetails AS
SELECT
    m.movieID,
    m.movieTitle,
    m.genre,
    m.releaseYear,
    d.directorName,
    ROUND(AVG(r.rating), 1) AS avgRating
FROM movies m
JOIN directors d ON m.directorID = d.directorID
JOIN reviews r ON m.movieID = r.movieID
WHERE m.releaseYear >= 2015
GROUP BY m.movieID, m.movieTitle, m.genre, m.releaseYear, d.directorName;
```

	MOVIEID	MOVIETITLE	GENRE	RELEASE_YEAR	DIRECTORNAME	AVGRATING
1	629556	The Diversity	War	2016	Nicole Reed	8.3
2	133347	Apostate Extraction	Mystery	2019	Dominique Smith	5.9
3	316141	Chromium	Drama	2019	Jonathan Ho	6.5
4	922796	Acute Medication	Sci-Fi	2025	Paige Castaneda	8.6
5	620864	The Combatant	Musical	2015	Sarah Perez	6
6	441992	The Testicle	Western	2017	Rebecca Mclaughlin	7
7	963203	Monograph	Music	2025	Heather Brown	6.3
8	962380	The Oyabun	Animation	2017	Sydney Marshall	4.5
9	894542	Sociolinguistic Sportswriter	Sci-Fi	2018	Michael Mcclain	4.7
10	126796	The Bleary Magpie	Film-Noir	2018	Joshua Edwards	5.8
11	492742	The Sentence	Short	2023	Steve Cook	7.9
12	718724	Certified Thought	Thriller	2022	Jason Martinez	7.9
13	898696	Katabolic Velocity	Horror	2019	Mr. John Lucero	7.3
14	168519	Missing Parable	Sci-Fi	2017	Katherine Contreras	6.5
15	995533	Observant Stifling	Drama	2024	Tyler Raymond	6.8
16	802293	The Cc Drab	Sci-Fi	2017	Karen Lindsey	3
17	436724	Craftsman	Romance	2018	Shelly Ford	7.1
18	888205	The Ingratiating Ranger	Short	2018	Lawrence Foster	5
19	187315	G	Drama	2025	Doris Carson	6.7
20	285505	Armour	Musical	2019	Valerie Lambert	6.4

Figure 6: Output of viewMovieDetails.

View 2: viewPopularMovies. This view also joins `movies` and `reviews`. It shows the movie id, title, genre, number of reviews, and average rating. It filter the database in a way that it displays only movies that has more than 10 reviews and also its average rating is more than 6. So the result only contains movies that are both reviewed enough and rated well.

```
CREATE OR REPLACE VIEW viewPopularMovies AS
SELECT
    m.movieID,
    m.movieTitle,
    m.genre,
    COUNT(r.reviewID) AS reviewCount,
    ROUND(AVG(r.rating), 1) AS avgRating
FROM movies m
JOIN reviews r ON m.movieID = r.movieID
WHERE m.movieID IN (
    SELECT movieID FROM reviews
    GROUP BY movieID
    HAVING COUNT(*) > 10
)
GROUP BY m.movieID, m.movieTitle, m.genre
HAVING ROUND(AVG(r.rating), 1) > 6;
```

	MOVIEID	MOVIETITLE	GENRE	REVIEWCOUNT	AVGRATING
1	521378	The Valued Massage	Biography	18	6.4
2	931258	The Turpentine	Musical	17	7.1
3	208114	Luminous Vertical	Animation	19	8.2
4	760733	The Unsweetened Butchery	Western	25	7.7
5	647417	The Subjective Submission	Mystery	22	7
6	880163	The Unresentful Megaton	Romance	11	7.2
7	787546	Grouchy Soaring	Musical	28	6.5
8	311944	Injured Sweep	Crime	30	7
9	839479	The Willow	Horror	20	7.3
10	742655	The Student	Animation	21	6.3
11	631095	The Restraint	Adventure	19	7.5
12	674275	Inexplicit Basso	Mystery	17	7.5
13	532996	Ethic	Sport	25	6.5
14	492742	The Sentence	Short	27	7.9
15	766516	The Latent Purification	Drama	29	7
16	641172	Sensory Commotion	Action	22	7
17	488680	The Ghostly Barker	Adventure	29	8.4
18	571829	Monarchal Appearing	Animation	23	6.2
19	788616	The Campaign	Biography	23	7
20	430566	Recumbent Shock	Romance	24	6.9

Figure 7: Output of viewPopularMovies.

Phase 8: PL/SQL Function and Procedure

In this phase I wrote one PL/SQL function and one PL/SQL procedure. The function is called `getAvgRating`. It takes a movie id as input and returns the average rating for that movie. Inside the function I declare a cursor that selects all ratings from the `reviews` table where the movie id matches. Then I loop through the cursor, add each rating to a running total, and count how many ratings there are. At the end, if the count is zero I return 0 so the function does not crash on movies with no reviews. Otherwise I divide the total by the count and round the result to one decimal before returning it.

```
CREATE OR REPLACE FUNCTION getAvgRating(targetMovieID IN NUMBER)
RETURN NUMBER IS
    calculatedAvg NUMBER := 0;
    ratingSum NUMBER := 0;
    ratingCount NUMBER := 0;

    CURSOR ratingsCursor IS
        SELECT rating FROM reviews WHERE movieID = targetMovieID;
BEGIN
    FOR ratingRow IN ratingsCursor LOOP
        ratingSum := ratingSum + ratingRow.rating;
        ratingCount := ratingCount + 1;
```

```
    END LOOP;

    IF ratingCount = 0 THEN
        RETURN 0;
    ELSE
        calculatedAvg := ratingSum / ratingCount;
        RETURN ROUND(calculatedAvg, 1);
    END IF;
END;
/
```

The procedure is called `auditLowRatedMovies`. It takes a threshold number as input. It uses a cursor to go through every row in the `movies` table. For each movie it calls `getAvgRating` to get the average. If the average is below the threshold and greater than zero, it inserts that movie into `movie_audit` with its id, title, and average rating. The trigger from Phase 5 then fills in the `auditID` automatically using the sequence. The procedure also starts by deleting old audit rows for movies whose average has gone up above the threshold, so the audit table stays up to date. At the end it commits all changes.

```
CREATE OR REPLACE PROCEDURE auditLowRatedMovies(ratingThreshold IN NUMBER)
     IS
    movieAvg NUMBER := 0;

    CURSOR moviesCursor IS
        SELECT movieID, movieTitle FROM movies;
BEGIN
    DELETE FROM movie_audit
    WHERE movieID IN (
        SELECT movieID FROM reviews
        GROUP BY movieID
        HAVING ROUND(AVG(rating), 1) >= ratingThreshold
    );

    FOR movieRow IN moviesCursor LOOP
        movieAvg := getAvgRating(movieRow.movieID);

        IF movieAvg < ratingThreshold AND movieAvg > 0 THEN
            INSERT INTO movie_audit (movieID, movieTitle, avgRating)
            VALUES (movieRow.movieID, movieRow.movieTitle, movieAvg);
        END IF;
    END LOOP;

    COMMIT;
END;
/
```

To test the function I ran it on one movie id directly:

```
SELECT getAvgRating(944368) FROM dual;
-- returns 5.3
```

To test the procedure I called it with a threshold of 2.0 and then selected everything from `movie_audit` to see the result:

```
EXEC auditLowRatedMovies(2.0);
SELECT * FROM movie_audit;
```

The output in Figure 8 shows the rows that were inserted into `movie_audit`. Each row has an auto-generated `auditID`, the movie id, the movie title, the average rating, and the audit date filled in by `SYSDATE`.

	AUDITID	MOVIEID	MOVIETITLE	AVGRATING	AUDITDATE
1	1	284903	Flowered Stucco	1.8	04-APR-26
2	2	577999	The Socialist	1.8	04-APR-26
3	3	825716	The Mercenary	1	04-APR-26
4	4	384952	The Exchanged Depersonalization	1.8	04-APR-26
5	5	643548	The Corps	1.5	04-APR-26
6	6	464978	The Controlled Purity	1.7	04-APR-26
7	7	896305	The Illegitimate	1.8	04-APR-26
8	8	325446	The Counterfactual Representation	1.9	04-APR-26
9	9	169200	The Easing	1.8	04-APR-26
10	10	607930	The Accessory Prexy	1	04-APR-26
11	11	583937	The Lingerie	1.5	04-APR-26
12	12	574330	The Monolingual Conception	1.9	04-APR-26
13	13	756352	The Megalomanic Champion	1	04-APR-26
14	14	451948	Nocent Approximation	1.5	04-APR-26
15	15	552278	Corrupted Turpentine	1.8	04-APR-26

Figure 8: Output of movie_audit.

Phase 9: Grants

In this phase, I gave the access to my instructor for the database objects I created. All grants go to the user `lkpeter`. I gave grants to all seven tables so the instructor can read the data:

```
GRANT SELECT ON directors TO lkpeter;
GRANT SELECT ON actors TO lkpeter;
GRANT SELECT ON users TO lkpeter;
GRANT SELECT ON movies TO lkpeter;
GRANT SELECT ON roles TO lkpeter;
GRANT SELECT ON reviews TO lkpeter;
GRANT SELECT ON movie_audit TO lkpeter;
```

I also gave grants to the two views so the instructor can query them directly:

```
GRANT SELECT ON viewMovieDetails TO lkpeter;
GRANT SELECT ON viewPopularMovies TO lkpeter;
```

The sequence also needs a grant so the instructor can see its current value:

```
GRANT SELECT ON auditSeq TO lkpeter;
```

Finally, I gave grants on the function and the procedure so the instructor can run them:

```
GRANT EXECUTE ON getAvgRating TO lkpeter;
GRANT EXECUTE ON auditLowRatedMovies TO lkpeter;
```

All grant statements ran without errors.

Language Model Usage

I used language models at several points during this project. This section describes each usage: which model I used, what I asked, how I used the output, and how I checked it.

Choosing a dataset theme.
Model: GPT-5.
I was not sure what kind of database to build, so I provided the task requirements to GPT-5 and asked:

> *What kind of datasets can I create? Give me possible options so I can choose between them.*

It gave me a list of possible domains with their tables, including online shopping, university system, social media platform, food delivery, library system, and movie database. I chose the movie database because I thought it would be easier to generate artificially using Python. I validated the output by reading through the options myself and picking the one that fit the task requirements.

Deciding the table structure.
Model: GPT-5.
After choosing the movie domain, I asked:

> *I liked the movie dataset. How should its structure be? The tables, the columns, the counts, and so on.*

It told me which tables to create, what columns each table should have, the order I need to create them in, and how to connect them with foreign keys. I used this as a guide to plan my schema. I validated it by checking that all the task requirements (minimum five tables, at least two big tables, at least one partitioned table) were covered.

Checking data quality.
Model: Claude Sonnet 4.6.
While testing the generated data, I found a duplicate primary key in one of the tables. After fixing it, I was worried there could be more similar problems that I could not find manually. I asked Claude:

> *I wrote scripts for synthetic data generation and I want a Jupyter notebook to check their quality. So I will be sure that the database is as desired.*

I also shared my generation scripts so it would understand the context. It produced a notebook that checks row counts, null values, duplicate rows, column ranges, and primary and foreign key consistency. I ran every check myself on the actual generated files and fixed any issues I found.

Verifying the reviews table creation.
Model: Claude Sonnet 4.6.
Before running the table creation script on the Aramis server, I sent the queries to Claude to check for mistakes. It pointed out that I was missing the line:

```
ALTER TABLE reviews ADD CONSTRAINT reviewsPk PRIMARY KEY (reviewID,
    reviewDate) USING INDEX LOCAL;
```

It explained that because the table is partitioned by reviewDate, Oracle requires the partition key to be part of the primary key. The LOCAL keyword tells Oracle to create a local index, which means the index is partitioned the same way as the table. I validated this by running the complete script and confirming that no errors appeared.

Fixing execution plan issues with indexes.
Model: Claude Sonnet 4.6.
After creating indexes, I noticed that both execution plans (before and after indexes) looked the same. I could not figure out why the indexes were not being used. I shared my script and the execution plan output with Claude and asked:

> *Why are both of these tables the same? Should not the indexes make the process faster?*

It suggested several fixes but they did not work. After going back and forth, it eventually told me to add this line before running the explain plan:

```
EXEC DBMS_STATS.GATHER_SCHEMA_STATS(USER);
```

This updates Oracle's internal statistics about the data, such as how many rows each table has. Without this, Oracle was using outdated statistics and

ignoring the indexes. After adding this line the execution plan changed and the cost dropped from 1389 to 24. I validated by running both explain plans myself and comparing the output.

Fixing grammatical issues on documentation.
Model: DeepSeek
In order to provide a smooth flow and correctness on my report, after writing each section I send it to the DeepSeek and ask it

> *Can you fix the grammatical issues on the provided text. Do not change the logic, sentences and words if it is not too serious problem. Return the fixed text inside a code cell.*

It returns a grammatically correct version of my text and I replace it with my version, because my version mostly looks messy and full with grammatical issues.

Reviewing the final documentation.
Model: DeepSeek. After finishing the documentation, I sent the project description and the full document to DeepSeek and asked it to check whether I had covered everything the task required. I chose DeepSeek because it does not restrict the input size and my document was around 15 to 20 pages.

NOTE: To view the data generation codes, tables, queries, and full documentation in one place, see this GitHub repository.